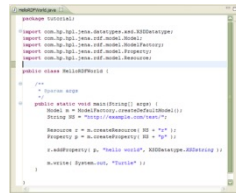


Translate to Intermediate Code

Chapter 7

Where Are We

Source Code



Lexical Analysis

Parsing

Semantic Analysis

IR Generation

IR Optimization

Code Generation

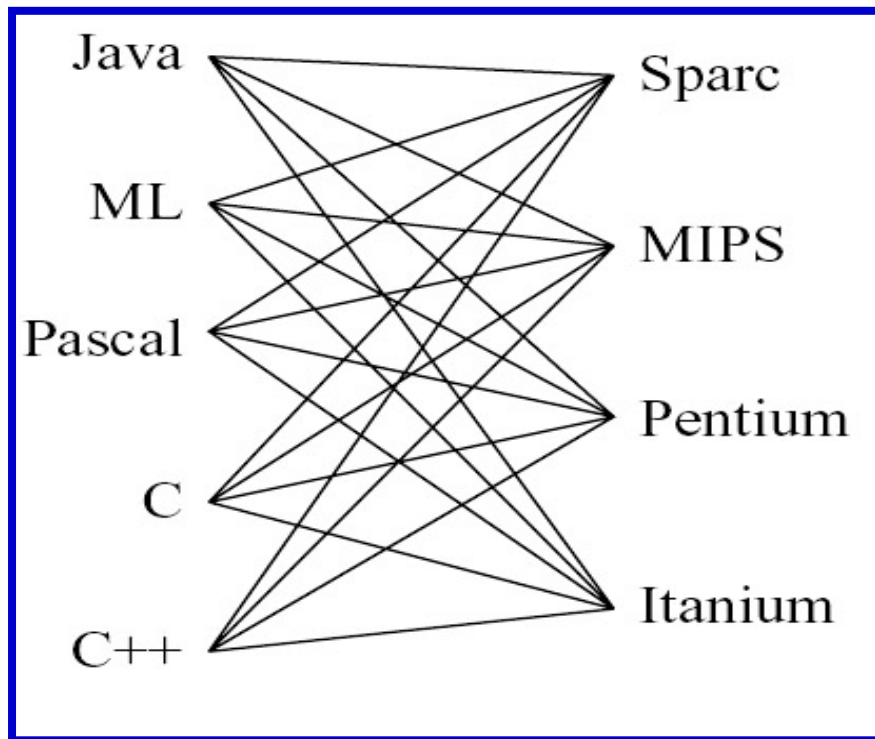
Optimization

[illegible]

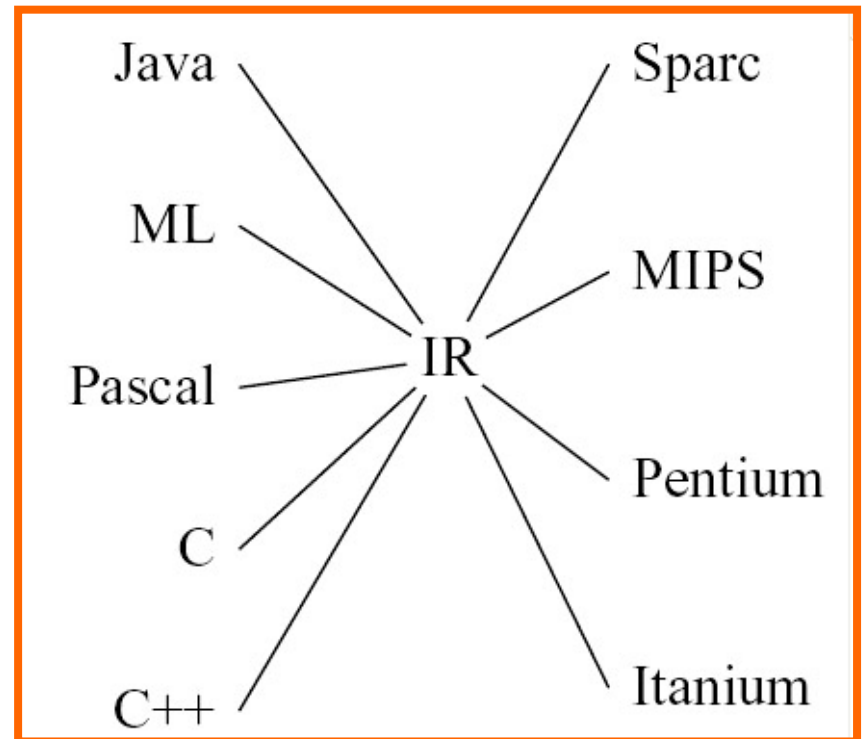
Machine Code

Motivation

- Why do we need intermediate representation (IR) ?
- Why not directly translate the AST to real machine code?
 - Modularity
 - Portability



$N * M$



$N + M$

What is Intermediate Representation?

- An intermediate representation (IR) is a kind of **abstract machine language**
 - Express the target-machine operations without committing to too much machine-specific details.
 - Be independent of the details of the source language
- Many different kinds of IR are used in compilers:
 - Tiger Compiler: expression trees
 - Three-Address Code
 - Static Single Assignment (SSA)
 - ...
- A compiler may use multi-level IR

Basic Concept

- The **front end**:
 - Lexical analysis
 - Parsing
 - Semantic analysis
 - Translation to **intermediate representation (IR)**
- The **back end**:
 - IR Optimization
 - Translation into machine language

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Outline

- **Three-Address Code**
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Three-Address Code

- The most **basic instruction** of three address code:

$$x = y \text{ op } z$$

- There can be four fields in an **instruction**
 - one for the operation
 - three for the **addresses**
- An address can be one of the following:
 - a **name**: a source-program name
 - a **constant**: e.g., 3
 - a **compiler-generated temporary**: e.g., t1

Three-Address Code

- There is **at most one operator** on the right side of an instruction.
- Thus a source language expression might be translated into the sequence of three-address instructions:

$$2 * a + (b - 3) \quad \longrightarrow \quad \begin{array}{l} t1 = 2 * a \\ t2 = b - 3 \\ t3 = t1 + t2 \end{array}$$

- It is necessary to **vary the form** of three-address code for different constructs in a programming language, e.g.,

$$t2 = - t1$$

- **No standard form** for three-address code exists:
 - One of the reasons: new forms need to be invented to express unusual features in a language

Three Address Code - Example

High-level language

```
read x
if 0 < x then
  fact:=1;
  repeat
    fact:=fact*x;
    x:=x-1
  until x=0;
  write fact
end
```

Three-address code

```
read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x==0
if_false t4 goto L2
write fact
label L1
halt
```

Three Address Code - Example

High-level language

```
read x
if 0 < x then
  fact:=1;
  repeat
    fact:=fact*x;
    x:=x-1
  until x=0;
  write fact
end
```

Three-address code

```
read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x==0
if_false t4 goto L2
write fact
label L1
halt
```

Three Address Code - Example

High-level language

```
read x
if 0 < x then
  fact:=1;
  repeat
    fact:=fact*x;
    x:=x-1
  until x=0;
  write fact
end
```

Three-address code

```
read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x==0
if_false t4 goto L2
write fact
label L1
halt
```

Three Address Code - Example

High-level language

```
read x
if 0 < x then
  fact:=1;
  repeat
    fact:=fact*x;
    x:=x-1
  until x=0;
  write fact
end
```

Three-address code

```
read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x==0
if_false t4 goto L2
write fact
label L1
halt
```

Three Address Code - Example

High-level language

```
read x
if 0 < x then
  fact:=1;
  repeat
    fact:=fact*x;
    x:=x-1
  until x=0;
  write fact
end
```

Three-address code

```
read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x==0
if_false t4 goto L2
write fact
label L1
halt
```

Three-Address Code - Implementation

- The entire sequence of three-address instructions is implemented as an array or linked list
- The most common implementation is to implement three-address code as **quadruples**:
 - one field for the operation
 - three fields for the addresses
- For those instructions that need fewer than three addresses, one or more of the address fields is given a null or “empty” value.

t1=x>0	(gt, x, 0, t1)
if_false t1 goto L1	(if_f, t1, L1, _)
fact=1	(asn, 1, fact, _)
label L2	(lab, L2, _, _)

- Other implementation: **triples**, **indirect triples**.

Outline

- Three-Address Code
- **Intermediate Representation Tree**
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Intermediate Representation Trees

A good intermediate representation has several qualities:

- Convenient for the semantic analysis phase to **produce**
- Convenient to **translate** into all real machine language
- Each construct must have a clear and **simple meaning**
 - IR **optimizing transformations** can easily be specified and implemented

Intermediate Representation Trees

- Individual pieces of abstract syntax can be **complicated** things
 - e.g., array subscripts, procedure calls.
- Individual “real machine” instructions can also have a **complicated** effect.
- **Unfortunately**, it is **not always the case** that complex pieces of the abstract syntax correspond exactly to the complex instructions that a machine can execute.
- **Therefore**, IR should have individual components that describe **only extremely simple things**: a single fetch, store, add, move, or jump.
 - any “chunky” piece of abstract syntax => the right set of abstract machine instructions (IR instructions).
 - groups of abstract machine instructions => “real” machine instructions.

Intermediate Representation Trees

Expressions (T_exp): the computation of some value, possibly with side effects

CONST(<i>i</i>)	The integer constant <i>i</i>
NAME(<i>n</i>)	The symbolic constant <i>n</i> (assembly language label)
TEMP(<i>t</i>)	Temporary <i>t</i> . i.e., virtual register
BINOP(<i>o</i>, <i>e1</i>, <i>e2</i>)	The application of binary operator <i>o</i> to operands <i>e1</i>, <i>e2</i>. The integer arithmetic operators: PLUS, MINUS, MUL, DIV; the integer bitwise logical operators: AND, OR, XOR; the integer logical shift operators: LSHIFT, RSHIFT; the integer arithmetic right-shift: ARSHIFT.
MEM(<i>e</i>)	The contents of <i>wordSize</i> bytes of memory starting at address <i>e</i>. When MEM is used as the left child of a MOVE, it means "store", but anywhere else it means "fetch".
CALL(<i>f</i>, <i>l</i>)	A procedure call: the application of function <i>f</i> to argument list <i>l</i>. Evaluate from left to right.
ESEQ(<i>s</i>, <i>e</i>)	The statement <i>s</i> is evaluated for side effects, then <i>e</i> is evaluated for a result.

Intermediate Representation Trees

Statements (T_{stm}): performs side effects and control flow – no return value

MOVE(TEMP t , e) Evaluate e and move it into temporary t .

MOVE(MEM(e_1), e_2) Evaluate e_1 , yielding address a . Then evaluate e_2 , and store the result into *wordSize* bytes of memory starting at a .

EXP(e) Evaluate e and discard the result.

JUMP(e , $labs$) Transfer control (jump) to address e . The destination e may be a literal label, as in **NAME(lab)**, or it may be an address calculated by any other kind of expression.

CJUMP(o , e_1 , e_2 , t , f) Evaluate e_1 , e_2 in that order, yielding values a , b . Then compare a , b using the relational operator o . If the result is true, jump to t ; otherwise jump to f .

SEQ(s_1 , s_2) The statement s_1 followed by s_2 .

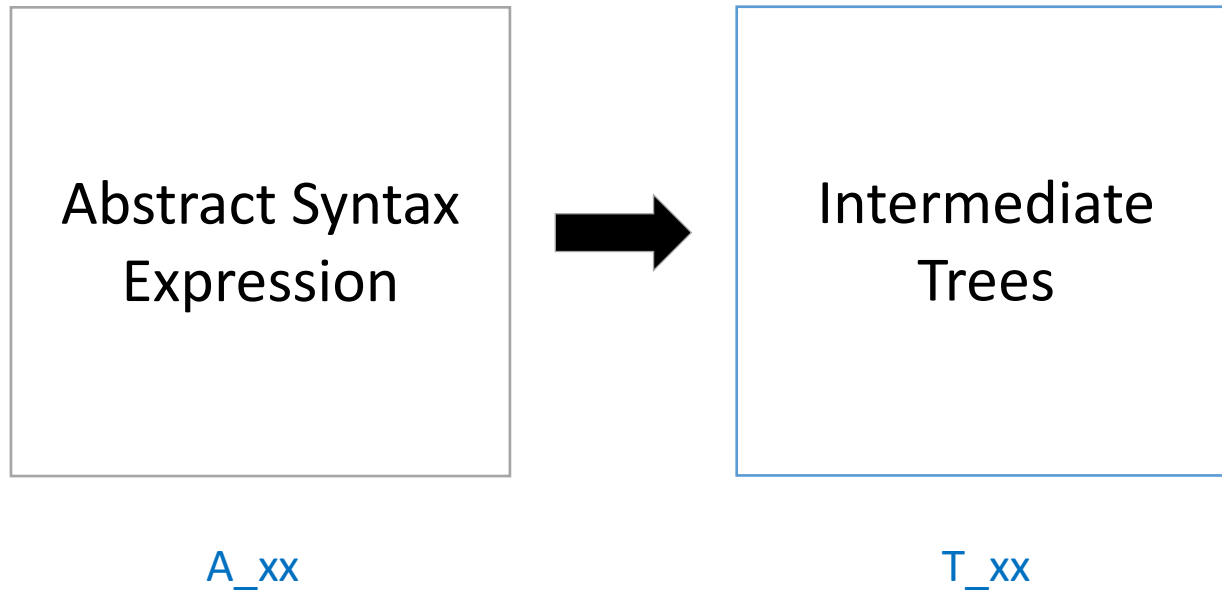
LABEL(n) Define the constant value of name n to be the current machine code address.

Name(n) uses symbol n , *Label(n)* defines symbol n

Outline

- Three-Address Code
- Intermediate Representation Tree
- **Translation into IR Trees**
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Translation into Trees



Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - **Expressions**
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
 - Translation of Declarations
 - Variable Definition
 - Function Definition

Kinds of Expressions

- What should the representation of an abstract syntax expression A_exp be in the language?
- Expressions with **return values**:
 - T_exp
- Expressions that **return no value** (such as some procedure calls, or while expressions)
 - T_stm
- Expressions with **Boolean values**, such as $a > b$
 - a conditional jump

Kinds of Expressions

- We model the three kinds expressions
 - **Ex**: an “expression”, a Tree expression
 - **Nx**: “no result”, a Tree statement
 - **Cx**: “conditional”, a Tree statement that may jump to a true-label or false-label

```
typedef struct Tr_exp_ *Tr_exp;
struct Cx { patchList trues; patchList falses; T_stm stm;};
struct Tr_exp_ {
    enum { Tr_ex, Tr_nx, Tr_cx } kind;
    union {
        T_exp ex;
        T_stm nx;
        struct Cx cx; } u;
};
static Tr_exp Tr_Ex ( T_exp ex);
static Tr_exp Tr_Nx ( T_exp nx);
static Tr_exp Tr_Cx ( patchList trues, patchList falses, T_stm
stm);
```

Tr_*: Translate module
T_*: Tree module

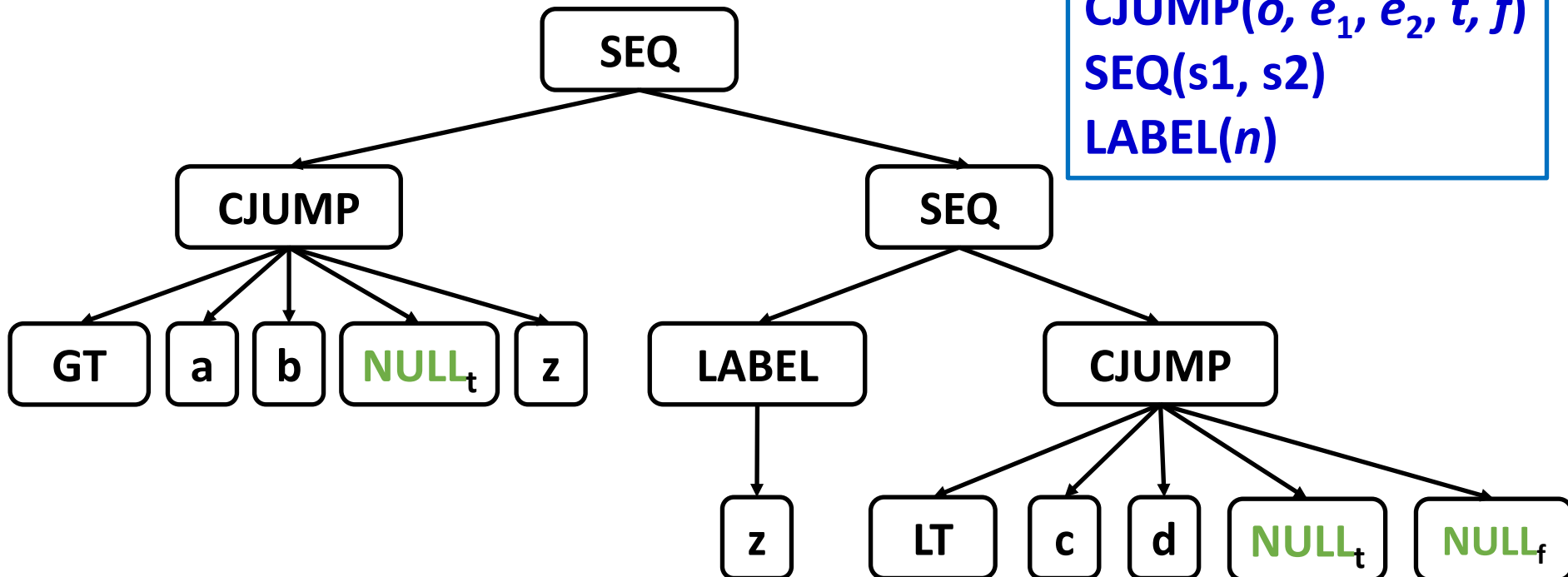
Kinds of Expressions

$a > b \mid c < d$

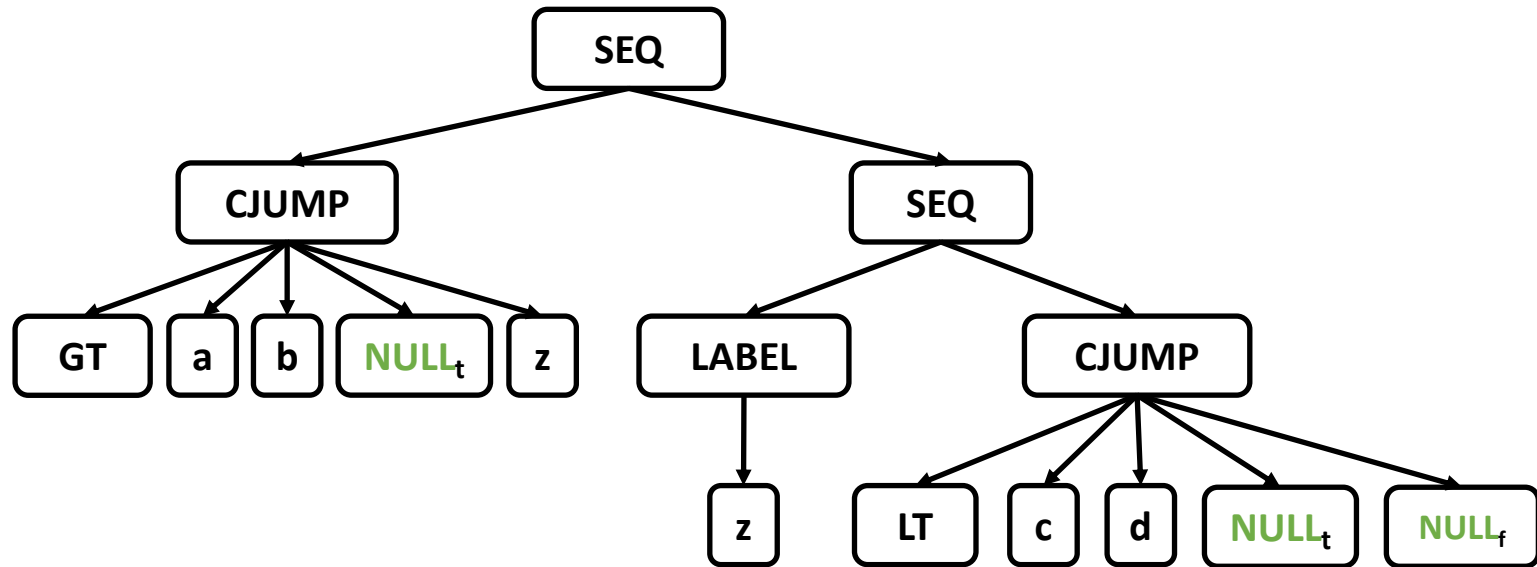


```
Temp_label z = Temp_newlabel ( );  
T_stm s1 = T_Seq(T_Cjump(T_gt,a,b, NULLt, z),  
                T_Seq (T_Label (z),  
                      T_Cjump (T_lt,c,d, NULLt, NULLf )));
```

$CJUMP(o, e_1, e_2, t, f)$
 $SEQ(s1, s2)$
 $LABEL(n)$



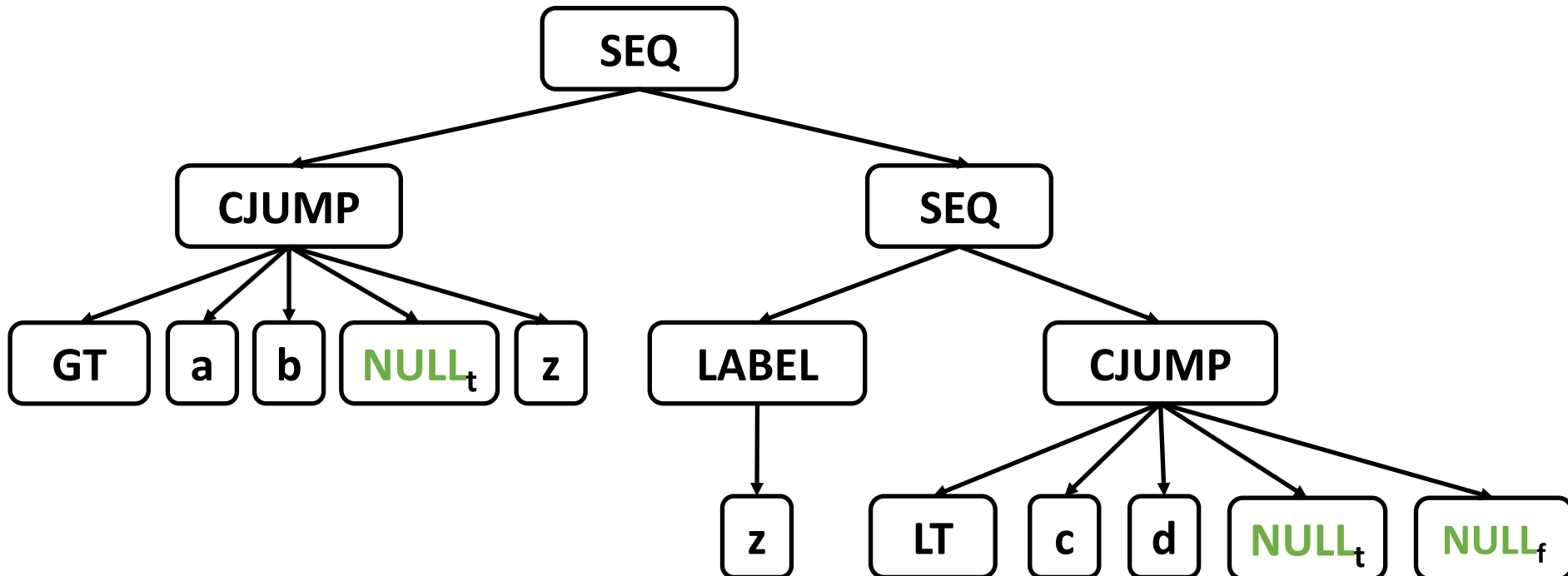
Kinds of Expressions



- We won't know the true-destination and false-destination until much later.
- We make **a list of places** which are now filled in with NULL and need to be **filled in with t** when t is known, and another list of all the places that need to be **filled in with f**.
 - **Backpatching**
 - **True patch list** and **false patch list**

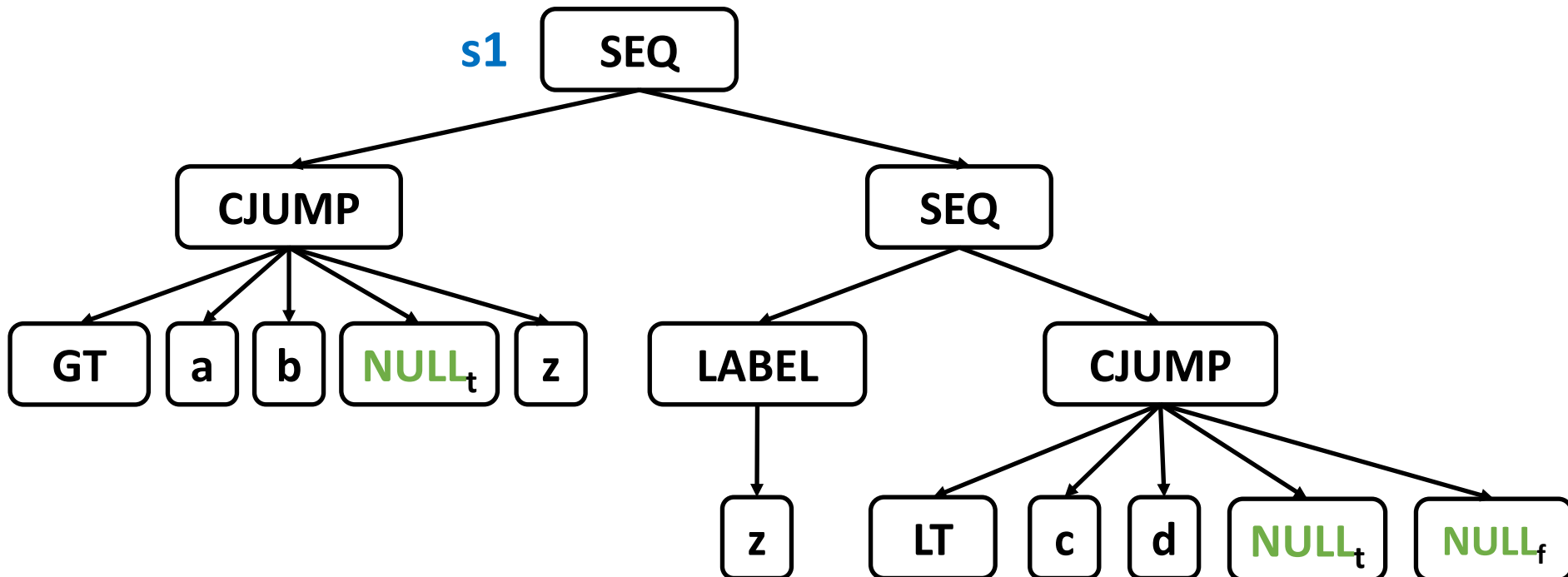
Kinds of Expressions

```
typedef struct patchList_ * patchList;  
struct patchList_ { Temp_label *head; patchList tail; };  
static patchList patchList(Temp_label *head, patchList tail);
```



Kinds of Expressions

```
patchList trues = PatchList(  
    &s1->u.SEQ.left->u.CJUMP.true,  
    PatchList(&s1->u.SEQ.right->u.SEQ.right->u.CJUMP.true,  
    NULL));  
patchList falses = PatchList(  
    &s1->u.SEQ.right->u.SEQ.right->u.CJUMP.false, NULL);  
Tr_exp e1 = Tr_Cx (trues, falses, s1);
```



Kinds of Expressions

- Sometimes we need to convert an expression of one kind to an equivalent expression of another kind.
- $flag := (a > b \mid c < d)$
 - requires the conversion of a Cx into an Ex

```
static T_exp unEx(Tr_exp e);
static T_stm unNx(Tr_exp e);
static struct Cx unCx(Tr_exp e);
```

- Three conversion functions
 - `Tr_exp` means the input expression can be of any kind
 - For different kinds of output expressions, we use different conversion functions.

$flag := (a > b \mid c < d) \quad \longrightarrow \quad \begin{array}{l} e = \text{Tr_Cx}(\text{trues}, \text{falses}, \text{stm}) \\ \text{MOVE}(\text{TEMP}(flag), \text{unEx}(e)) \end{array}$

Kinds of Expressions

```
static T_exp unEx(Tr_exp e) {
switch (e->kind) {
case Tr_ex:
    return e->u.ex;
case Tr_cx: {
    Temp_temp r = Temp_newtemp( );
    Temp_label t = Temp_newlabel( ), f= Temp_newlabel( );
    doPatch(e->u.cx.trues, t);
    doPatch(e->u.cx.falses, f);
    return T_Eseq(T_move(T_Temp(r), T_Const(1)),
        T_Eseq(e->u.cx.stm, T_Eseq(T_Label(f),
            T_Eseq(T_Move(T_Temp(r), T_Const(0)),
                T_Eseq(T_Label(t), T_Temp(r))))));
    }
case Tr_nx:
    return T_Eseq(e->u.nx, T_Const(0));
}
assert(0);
}
```

Kinds of Expressions

```
static T_exp unEx(Tr_exp e) {
switch (e->kind) {
case Tr_ex:
    return e->u.ex;
case Tr_cx: {
    Temp_temp r = Temp_newtemp( );
    Temp_label t = Temp_newlabel( ), f= Temp_newlabel( );
    doPatch(e->u.cx.trues, t);
    doPatch(e->u.cx.falses, f);
    return T_Eseq(T_move(T_Temp(r), T_Const(1)),
                  T_Eseq(e->u.cx.stm, T_Eseq(T_Label(f),
                  T_Eseq(T_Move(T_Temp(r), T_Const(0)),
                  T_Eseq(T_Label(t), T_Temp(r))))));
}
case Tr_nx:
    return T_Eseq(e->u.nx, T_Const(0));
}
assert(0);
}
```

Kinds of Expressions

```
static T_exp unEx(Tr_exp e) {  
    switch (e->kind) {  
        case Tr_ex:  
            return e->u.ex;  
        case Tr_cx: {  
            Temp_temp r = Temp_newtemp( );  
            Temp_label t = Temp_newlabel( ), f= Temp_newlabel( );  
            doPatch(e->u.cx.trues, t);  
            doPatch(e->u.cx.falses, f);  
            return T_Eseq(T_move(T_Temp(r), T_Const(1)),  
                          T_Eseq(e->u.cx.stm,  
                                T_Eseq(T_Label(f),  
                                      T_Eseq(T_Move(T_Temp(r), T_Const(0)),  
                                            T_Eseq(T_Label(t), T_Temp(r))))));  
        }  
        case Tr_nx:  
            return T_Eseq(e->u.nx, T_Const(0));  
    }  
    assert(0);  
}
```

if Cx
 return 1
else
 return 0



MOVE(TEMP r, 1)
Cx
LABEL(f)
MOVE(TEMP r, 0)
LABEL(t)
TEMP(r)

Kinds of Expressions

$a > b \mid c < d$



```
Temp_label z = Temp_newlabel ( );  
T_stm s1 = T_Seq(T_Cjump(T_gt,a,b, NULLt, z),  
                T_Seq (T_Label (z),  
                      T_Cjump (T_lt,c,d, NULLt, NULLf )));
```

```
MOVE(TEMP r, 1)  
Cx  
LABEL(f)  
MOVE(TEMP r, 0)  
LABEL(t)  
TEMP(r)
```

$(a > b) \mid (c < d)$



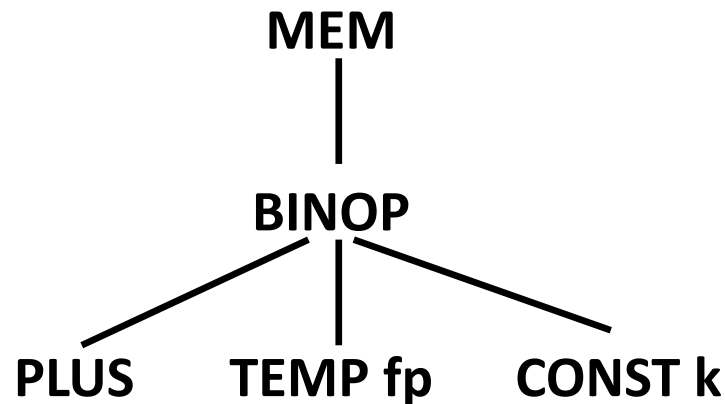
```
MOVE(TEMP r, 1)  
CJUMP(GT, a, b, t, z)  
LABEL(z)  
CJUMP(LT, c, d, t, f)  
LABEL(f)  
MOVE(TEMP r, 0)  
LABEL(t)  
TEMP(r)
```

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - **Simple Variables**
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Simple Variables

- Translate a simple variable v declared in the current procedure's stack frame:



MEM(BINOP(PLUS,TEMP fp, CONST k))

- k is the offset of v within the frame
- **TEMP fp**: the frame-pointer register
- For the Tiger compiler, all variables are the same size — the natural word size of the machine.

Simple Variables

The interface between **Semant** and **Translate**

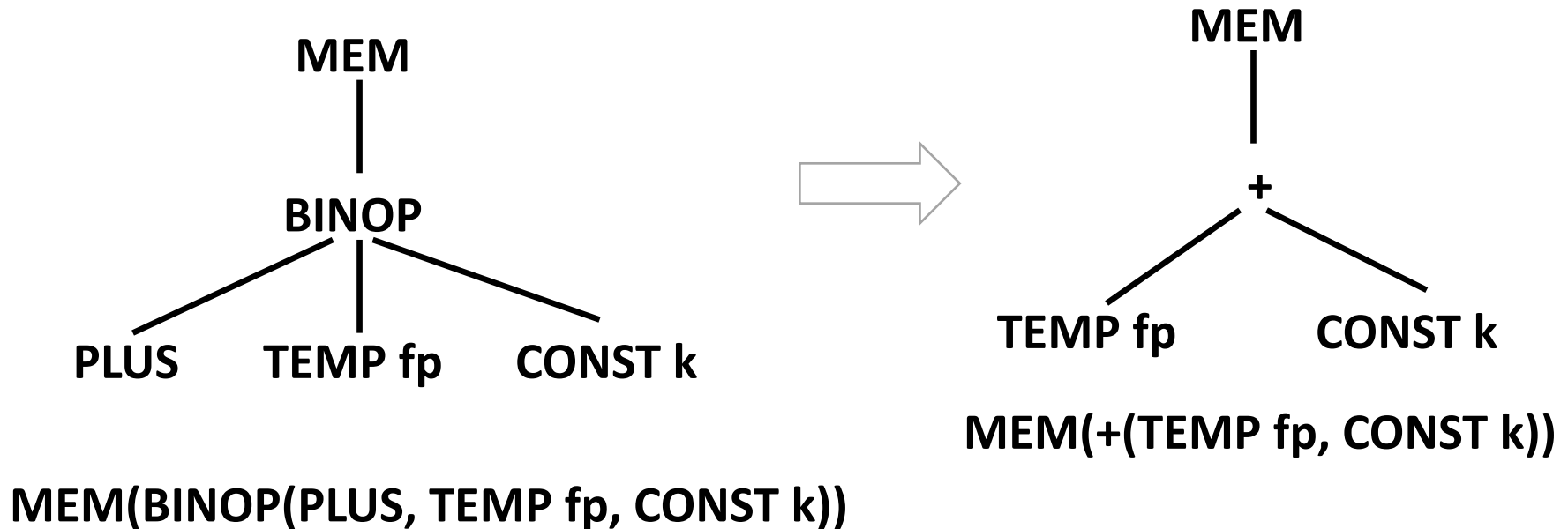
- The **Semant** module should not contain any direct reference to the **Tree** or **Frame** module
- Any manipulation of IR trees should be done by **Translate**

```
Tr_Exp Tr_simpleVar (Tr_Access, Tr_Level);
```

- **Semant** passes the access of *x* (obtained from *Tr_allocLocal*) and the level of the function in which *x* is used.

Simple Variables

- We will abbreviate `BINOP(PLUS, e1, e2)` as `+(e1, e2)`



Simple Variables

- To translate `v`, we need to know the `frame pointer` and the `word size`
 - Machine-dependent definitions
- We add a frame-pointer register `FP` and a constant whose value is the machine's natural `word size` into the Frame module.

```
/* frame.h */  
.....  
Temp_temp F_FP(void);  
extern const int F_wordSize;  
T_exp F_Exp(F_access acc, T_exp framePtr);
```

- The function `F_Exp` is used by `Translate` to turn a `F_access` into the `Tree expression`

Simple Variables

```
/* frame.h */
.....
Temp_temp F_FP(void);
extern const int F_wordSize;
T_exp F_Exp(F_access acc, T_exp framePtr);
```

Example

- An access a is `InFrame(k)`:

`F_Exp(a, T_Temp(F_FP()))`

Returns

`MEM(BINOP(PLUS,TEMP FP,CONST( $k$ )))`

- An access a is `InReg( $t_{832}$ )`
 - Simply return TEMP t_{832}

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - **Array Variables**
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Array Variables

- Different programming languages treat array-valued variables differently.
- In **Pascal**: An array variable stands for the **contents of the array**
 - The following snippet copies the contents of array b into array a (all 12 integers)

```
var a,b : array[1..12] of integer;  
begin  
    a:=b  
end.
```

- In **C**: arrays are like “pointer constants”

```
{ int a[12], b[12];  
  a=b;  
}
```

Illegal

```
{ int a[12], *b;  
  b=a;  
}
```

legal

*b points to the
beginning of the array a*

Array Variables

- In Tiger (as in Java and ML), array variables behave like pointers
 - has no named array constants as in C
 - new array values are created (and initialized) by the construct $t_a[n] \text{ of } i$
 - t_a is the name of an array type
 - n is the number of elements
 - i is the initial value of each element

```
let
  type intArray = array of int
  var a := intArray[12] of 0
  var b := intArray[12] of 7
in a := b
end
```

- a ends up pointing to the same 12 **sevens** as the variable b ;
- the original 12 **zeros** allocated for a are discarded.

Array Variables

- Tiger **record** values are also pointers.
 - **Record** assignment, like array assignment, is pointer assignment and does not copy all the fields.

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - **Structured L-Values**
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Structured L-Values

- **L-value**: the result of an expression that can occur on the *left* of an assignment statement.
 - x , $y.p$ or $a[i+2]$
 - denotes a location that can be assigned to
 - can occur on the right of an assignment statement
- **R-value**: the result of an expression that can only appear on the *right* of an assignment.
 - $a + 3$ or $f(x)$
 - **r-value** does not denote an assignable location

Structured L-Values

- An integer or pointer value is a “**scalar**”: it has only one component
- In **Tiger**:
 - All the **variables** and **l-values** are **scalar**
 - An array or record variable is really a pointer (a kind of **scalar**)
- In **C** or **Pascal**, there are structured l-values – **structs** in C, **arrays** and **records** in Pascal – that are **not scalar**
- To translate structured L-Values, update **T_Mem**:

```
T_exp T_Mem(T_exp, int size);
```

```
Mem(+(TEMP fp,CONST  $k_n$ ), S)
```

- **S** indicates the size of the object to be fetched or stored

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - **Subscripting and Field Selection**
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Subscripting and Field Selection

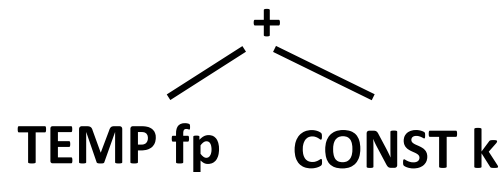
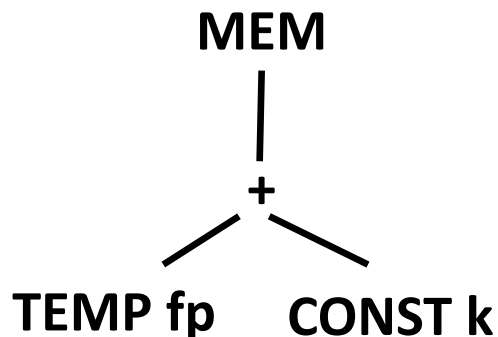
- To compute the address of $a[i]$:

$$(i - l) \times s + a$$

- a : the base address of the array elements
 - l : the lower bound of the index range
 - s : the size (in bytes) of each array element
-
- If a is global, with a compile-time constant address, the subtraction $a - s \times l$ can be done at compile time.
-
- To calculate the address of the field f of a record a
$$\text{offset}(f) + a$$

Subscripting and Field Selection

- An array variable **a** is an l-value, so is an array subscript expression **a[i]**
- To obtain the address of **a[i]**, we need to do arithmetic on the address of **a**.
- In a **Pascal** compiler,
 - If we translate the l-value **a** into something like the **left IR tree**, we **cannot do arithmetic** on the address of **a**.



- Therefore, we should translate the l-value **a** into the Tree expression representing its address, just like the **right IR tree**.

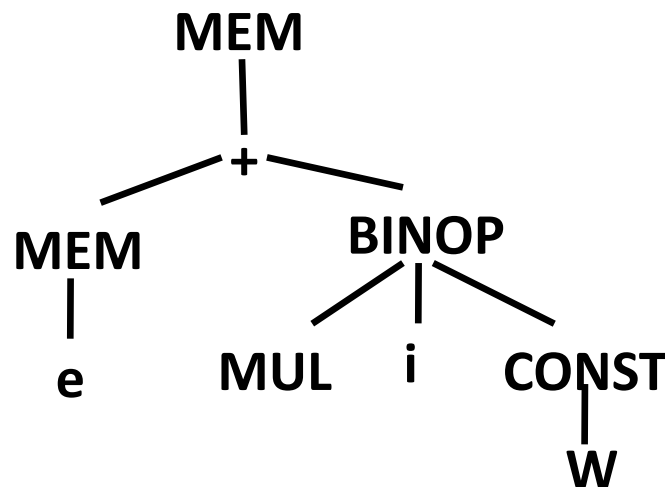
Subscripting and Field Selection

In a **Pascal** compiler, what could happen to this l-value **a**?

- A particular element might be **subscripted**, yielding a (smaller) l-value, e.g., **a[i]**. A “+” node would add $(i - l) \times s$ to **a**.
- The l-value (representing the entire array) might be used in a context where an r-value is required, e.g., **b = a**. Then the l-value is *coerced* into an r-value by applying the **MEM** operator to it.

Subscripting and Field Selection

- In the **Tiger** language, all **record** and **array** values are really **pointers** to record and array structures.
- The “**base address**” of the array is really the contents of a pointer variable, so **MEM is required**.
- IR tree of **a[i]** is



- **MEM(e)** represents **a**
- **W** is the word size

MEM(+ (MEM(e), BINOP(MUL, i, CONST W)))

Subscripting and Field Selection

- Technically, an l-value should be represented as an address (without the top **MEM** node)
 - converting an l-value to an r-value: **fetching from that address** (adding **MEM**)
 - assigning to an l-value: **storing to that address**
- We are attaching the MEM node to the l-value before knowing whether it is to be fetched or stored.
 - In the **Tree IR**, **MEM** means both store (when used as the left child of a **MOVE**) and fetch (when used elsewhere)

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - **Arithmetic**
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Arithmetic

In the **Tiger** Programming Language:

- Each integer arithmetic operator corresponds to a **Tree** operator.
- No unary arithmetic operators
 - Unary negation of integers: implemented as subtraction from zero; $-n \Rightarrow 0 - n$
 - Unary complement: implemented as XOR with all ones
- No floating point:
 - Unary floating-point negation cannot be implemented as subtraction from zero.
 - Many floating-point representations allow a negative zero
 - The negation of negative zero is positive zero, and vice versa
- The **Tree** language does not support unary negation very well

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - **Conditionals**
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Conditionals

- The result of a comparison operator will be a Cx expression
 - a statement (T_stm) s that will jump to any true-destination and false-destination
- Conditional expressions can be combined easily with Tiger operators & and |
 - e.g., $a > b \mid c < d$
- Therefore, an expression such as $x < 5$ will be translated as a Cx with:

```
stm = CJUMP (LT, x, CONST(5), NULLt, NULLf)  
trues = {t}  
falses = {f}
```

- e.g., $x < 5 \ \& \ y > 6$, $x < 5 \mid y > 6$

Conditionals

- How to handle if-expression?

if e_1 then e_2 else e_3

- The most straightforward thing:
 - e_1 : Cx expression;
 - e_2 and e_3 : Ex expressions
 - apply unCx to e_1
 - apply unEx to e_2 and e_3
 - make two labels t and f for the conditional
 - allocate a temporary r
 - after label t , move e_2 to r
 - after label f , move e_3 to r
 - both branches finish by jumping to a newly created “join” label

```
unCx( $e_1$ )  
LABEL  $t$   
 $r = \text{unEx}(\mathbf{e_2})$   
JUMP join  
LABEL  $f$   
 $r = \text{unEx}(\mathbf{e_3})$   
JUMP join  
...  
LABEL join  
TEMP  $r$   
...
```

Correct but not very efficient

Conditionals

if e_1 then e_2 else e_3

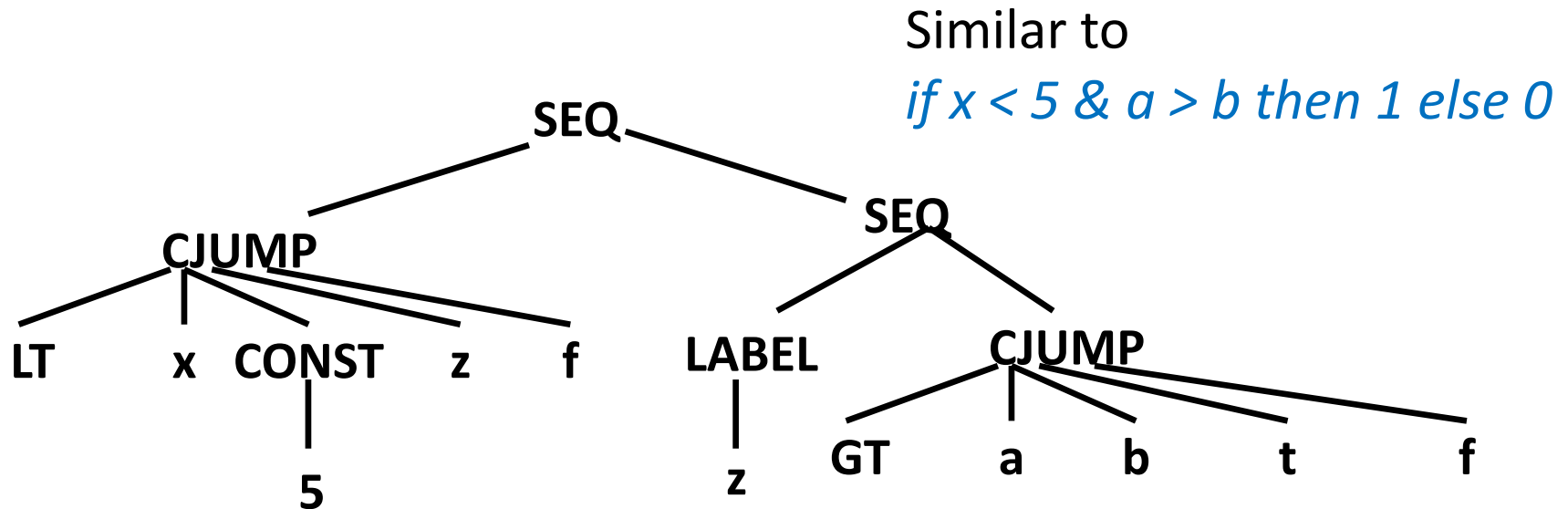
- If **e2** and **e3** are both “statements” (expressions that return no value), **unEx** will work, but it might be better to recognize this case specially.
- If **e2** or **e3** is a **Cx** expression, **unEx** will yield a horrible tangle of jumps and labels.
 - recognize this case specially

[illegible]

Conditionals

Example: *if $x < 5$ then $a > b$ else 0*

- $x < 5$ translates into $Cx(s1)$
- $a > b$ will be translated as $Cx(s2)$



$SEQ(s1(z,f), SEQ(LABEL\ Z, s2(t,f)))$

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - **While Loops**
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

While Loops

- The general layout of a **while** loop is:

```
test:
    if not(condition) goto done
    body
    goto test
done:
```

- If a **break** statement occurs within the body (and not nested within any interior **while** statements), the translation is simply a **JUMP** to *done*
 - How to know the **done** label?
 - *transExp* will have a new formal parameter **break** that is set to the *done* label of the nearest enclosing loop

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - **For Loops**
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

For Loops

- A straightforward approach to translate **for** statements
 - rewrite the *abstract syntax* into the abstract syntax of the **let/while** expression shown.

```
for i := lo to hi  
do body
```

```
let var i := lo  
    var limit := hi  
in while i <= limit  
    do (body; i := i+1)  
end
```

- Problem:
 - when *limit=maxint*, *i+1* will overflow
- How to solve this problem?

```
if lo > hi goto done  
i := lo  
limit := hi  
test:  
    body  
    if i >= limit goto done  
    i := i+1  
    goto test  
done:
```

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - **Function Call**
- Translation of Declarations
 - Variable Definition
 - Function Definition

Function Call

- Translating a function call $f(a_1, \dots, a_n)$ is simple, except that the **static link** must be added as an implicit extra argument:

$\text{CALL}(\text{NAME } l_f, [sl, e_1, e_2, \dots, e_n])$

- l_f : the label of f
- sl : the static link

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of Declarations**
 - Variable Definition
 - Function Definition

Declarations

The call to **transDec** will now side-effect the **frame** data structure:

- For each **variable declaration** within a function body
 - additional space (at **FP + offset**) will be reserved in the **frame**
- For each **function declaration**
 - a new “fragment” of Tree code will be kept for the function’s body

	<code>_global name</code>	
<code>name:</code>	<code>...</code>	prologue
	assembly code for body	
	<code>...</code>	epilogue

Variable Declarations

- The *transDec* function updates the *value environment* and **type environment**.
- For **variable initializations**:
 - *transDec* translates them to **assignment expressions**
 - The tree expressions of such assignment expressions must be put just before the **body** of the *let*.
 - *let ... in ... end*
- If *transDec* is applied to **function and type declarations**, the result will be a “no-op” expression, such as **EXP(CONST(0))**.

Function Declarations

- A function is translated into an “assembly language segment” with three components:
 - a **prologue**
 - a **body**
 - an **epilogue**

	<code>_global name</code>	
<code>name:</code>	<code>...</code>	prologue
	assembly code for body	
	<code>...</code>	epilogue

Function Declarations

- A **prologue** contains:
 1. pseudo-instructions to **mark the beginning of a function** (needed in the particular assembly language)
 2. a **label definition** of the function name
 3. an instruction to **adjust the stack pointer** (to allocate a new frame)
 4. instructions to **save “escaping” arguments** (including the static link) into the frame, and to **move nonescaping arguments** into fresh temporary registers;
 5. store instructions to **save any callee-save registers** (including the return address register) used within the function

Function Declarations

- The **body** of a Tiger function is an expression
 6. The translated expression
- An **epilogue** contains:
 7. an instruction to **move the return value** (result of the function) to the register
 8. load instructions to **restore the callee-save registers**
 9. an instruction to **reset the stack pointer** (to deallocate the frame)
 10. a **return** instruction (**JUMP** to the return address)
 11. pseudo-instructions, as needed, to **announce the end of a function**

Fragment

The **Translate** phase should produce a **fragment** for each function, which contains:

- **frame**: The frame descriptor containing machine-specific information about local variables and parameters
- **body**: The result returned from *procEntryExit1*

Outline

- Three-Address Code
- Intermediate Representation Tree
- Translation into IR Trees
 - Expressions
 - Simple Variables
 - Array Variables
 - Structured L-Values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- Translation of Declarations
 - Variable Definition
 - Function Definition

Homework

- **7.2**

Kinds of Expressions

```
void doPatch (patchList tList, Temp_label label) {  
    for ( ; tList; tList = tList->tail)  
        *(tList->head) = label;  
}
```

```
patchList joinPatch (patchList first, patchList second) {  
    if (!first) return second;  
    for (; first->tail; first = first->tail);  
    first->tail = second;  
    return first;  
}
```

Fragment

The *frag* datatype

```
/* frame.h */
...
typedef struct F_frag_ * F_frag;
struct F_frag_ {
    enum {F_stringFrag, F_procFrag} kind;
    union {
        struct {Temp_label label; string str;} stringg;
        struct {T_stm body; F_frame frame;} proc;
    } u;
};
F_frag F_StringFrag(Temp_label label, string str);
F_frag F_ProcFrag(T_stm body, F_frame frame);
typedef struct F_fragList_ *F_fragList;
struct F_fragList_ {F_frag head; F_fragList tail;};
F_fragList F_FragList(F_frag head, F_fragList tail);

/* translate.h */
...
void Tr_procEntryExit(Tr_level level, Tr_exp body,
Tr_accessList formals);
F_fragList Tr_getResult(void);
```

Fragment

The *frag* datatype

```
/* frame.h */
...
typedef struct F_frag_ * F_frag;
struct F_frag_ {
    enum {F_stringFrag, F_procFrag} kind;
    union {
        struct {Temp_label label; string str;} stringg;
        struct {T_stm body; F_frame frame;} proc;
    } u;
};
F_frag F_StringFrag(Temp_label label, string str);
F_frag F_ProcFrag(T_stm body, F_frame frame);
typedef struct F_fragList_ *F_fragList;
struct F_fragList_ {F_frag head; F_fragList tail;};
F_fragList F_FragList(F_frag head, F_fragList tail);

/* translate.h */
...
void Tr_procEntryExit(Tr_level level, Tr_exp body,
Tr_accessList formals);
F_fragList Tr_getResult(void);
```

Fragment

```
/* frame.h */
```

```
...
```

The *frag* datatype

```
typedef struct F_frag_ * F_frag;
struct F_frag_ {
    enum {F_stringFrag, F_procFrag} kind;
    union {
        struct {Temp_label label; string str;} stringg;
        struct {T_stm body; F_frame frame;} proc;
    } u;
};
F_frag F_StringFrag(Temp_label label, string str);
F_frag F_ProcFrag(T_stm body, F_frame frame);
typedef struct F_fragList_ *F_fragList;
struct F_fragList_ {F_frag head; F_fragList tail;};
F_fragList F_FragList(F_frag head, F_fragList tail);
```

```
/* translate.h */
```

```
...
```

```
void Tr_procEntryExit(Tr_level level, Tr_exp body,
Tr_accessList formals);
F_fragList Tr_getResult(void);
```

create a fragment for a function

return the list of all function fragments